CHAPTER – X

PYTHON CLASSES AND OBJECTS

10.1 :::: Introduction:

- Python is a Object Oriented Programming Language.
- Classes and Object are the key features of OOP.

- **Class** is a main building block in Python.
- **Object** is a collection of data and function that act on those data.
- **Objects** are called as Instances of a class.
- A class variable is called as **Object**.
- In Python, everything is an object.
- Integer variable is an object of Int class. String variable is an object of String class.

10.2 :::: Defining Classes:

- A class can be defined anywhere of the program.
- Class can be defined by using the keyword 'class'.
- Every class has unique class name and followed by colon (:).
- The variables defined within a class are called as **Class variables.**
- The functions within class are called as **Methods.**
- Class variables and Methods together called as **Members of the class.**
- Class members can be accessed by an object of the class.
- Example:

```
class sample:
    x, y=10, 20
```

10.3 :::: Creating Objects:

- The process of creating object is called as **Class Instantiation.**
- Syntax:

```
object_name=class_name( )
```

- Example:

```
class sample:
    x, y=10, 20

obj=sample( )        # class instantiation
print("X=",obj.x);
```

- Output:

```
X=10
```

10.4 :::: Accessing Class Members:

- Class members are accessed by using dot(.) operator.
- Syntax:

```
object_name . class_member
```

- Example:

```
class sample:
    x, y=10, 20
obj=sample( )        # class instantiation
print("X=",obj.x);
```

- Output:

```
X=10
```

10.5 :::: Class Methods:

- Method is similar to the ordinary function.
- Only difference is, class method must have an argument self.
- No need to pass value to self argument. Python provides value to self argument.
- Even if a method takes no argument, it is compulsory to specify the self argument.
- If a method takes one argument then it will take it as two argument such as self and defined argument.
- Example:

```
class student:
    x,y=10, 20          # class variable

    def process (self):
        sum=student.x + student.y
        print("Sum=",sum)
        return

obj=student( )
obj.process( )
```

- Output:

```
Sum=30
```

10.6 :::: Constructor and Destructor:

Constructor:

- Constructor is a special function.
- Constructor is automatically executed when an object of the class is created.
- In Python, **init function** is act as constructor
- **init function** should begins and ends with double underscore.
- **init function** can be defined with or without arguments.
- **init function (constructor)** is used to initialize the class variables.
- General Format:

```
def __init__ (self, [args]):
    # statements
```

- Example:

```
class sample:
    def __ init__(self, num):
        self.num=num
        print ("The value is ", num)
obj=sample(10)
```

- Output:

```
The value is 10
```

```
class sample :
    def __init__ (self,num) :
        self.num = num
        print (" The value is :", num)
obj = sample (10)
```

Destructor:

- Destructor is a special function.
- Destructor is automatically executed when an object exit from the scope.
- In Python, **del function** is act as destructor
- **dell function** should begins and ends with double underscore.
- **del function** can be defined without arguments.
- **del function (destructor)** is used to destroy the class variables.
- General Format:

```
def __del__ (self) :
    # statements
```

- Example:

```
class sample:
    def __ init__(self, num):
        self.num=num

    def __ del__(self):
        print ("The value is ", self.num)

obj=sample(10)
```

- Output:

```
The value is 10
```

10.7 :::: Public and Private data members:

Public Data Members:

- By default, variable defined within class is public.
- Public variables can be accessed anywhere in the program using dot operator.

Private Data Members:

- The variable prefixed with double underscore becomes private variable.
- Private variables can be accessed only within the class.
- Example:

```
class Sample:
    def __init__(self, n1, n2):
        self.n1=n1
        self.__n2=n2
    def display(self):
        print("Class variable 1 = ", self.n1)
        print("Class variable 2 = ", self.__n2)
S=Sample(12, 14)
S.display()
print("Value 1 = ", S.n1)
print("Value 2 = ", S.__n2)
```

Hands on Experience

1. Write a program using class to store name and marks of students in list and print total marks.
2. Write a program using class to accept three sides of a triangle and print its area.
3. Write a menu driven program to read, display, add and subtract two distances.

Evaluation

6MVJH

Part - I

Choose the best answer **(1 Mark)**

1. Which of the following are the key features of an Object Oriented Programming language?
   (a) Constructor and Classes (b) Constructor and Object
   (c) Classes and Objects (d) Constructor and Destructor
2. Functions defined inside a class:
   (a) Functions (b) Module
   (c) Methods (d) section
3. Class members are accessed through which operator?
   (a) & (b) .
   (c) # (d) %
4. Which of the following method is automatically executed when an object is created?
   (a) __object__( ) (b) __del__( )
   (c) __func__( ) (d) __init__( )
5. A private class variable is prefixed with
   (a) __ (b) &&
   (c) ## (d) **
6. Which of the following method is used as destructor?
   (a) __init__( ) (b) __dest__( )
   (c) __rem__( ) (d) __del__( )

7. Which of the following class declaration is correct?

(a) class class_name (b) class class_name<>

(c) class class_name: (d) class class_name[ ]

8. Which of the following is the output of the following program?

```
class Student:
        def __init__(self, name):
                self.name=name
                print (self.name)
S=Student("Tamil")
```

(a) Error (b) Tamil

(c) name (d) self

9. Which of the following is the private class variable?

(a) __num (b) ##num

(c) $$num (d) &&num

10. The process of creating an object is called as:

(a) Constructor (b) Destructor

(c) Initialize (d) Instantiation

Part -II

Answer the following questions (2)

1. What is class?
2. What is instantiation?
3. What is the output of the following program?

```
class Sample:
        __num=10
        def disp(self):
                print(self.__num)
S=Sample()
S.disp()
print(S.__num)
```

print (S.__num)

Attribute error: 'Sample' object has no attribute '__num'

4. How will you create constructor in Python?
5. What is the purpose of Destructor?

Part -III

Answer the following questions **(3 Marks)**

1. What are class members? How do you define it?
2. Write a class with two private class variables and print the sum using a method.
3. Find the error in the following program to get the given output?

```
class Fruits:
        def __init__(self, f1, f2):
                self.f1=f1
                self.f2=f2
        def display(self):
                print("Fruit 1 = %s, Fruit 2 = %s" %(self.f1, self.f2))
F = Fruits ('Apple', 'Mango')
del F.display  => Error This line is to be deleted for getting the expected result.
F.display()
```

Output

Fruit 1 = Apple, Fruit 2 = Mango

4. What is the output of the following program?

```
class Greeting:
        def __init__(self, name):
                self.__name = name
        def display(self):
                print("Good Morning ", self.__name)
obj=Greeting('Bindu Madhavan')
obj.display()
```

Good Morning Bindu Madhavan :

5. How to define constructor and destructor in Python?

Part -IV

Answer the following questions **(5 Marks)**

1. Explain about constructor and destructor with suitable example.

References

1. *https://docs.python.org/3/tutorial/index.html*
2. *https://www.techbeamers.com/python-tutorial-step-by-step/#tutorial-list*
3. *Python programming using problem solving approach – Reema Thareja – Oxford University press.*
4. *Python Crash Course – Eric Matthes – No starch press, San Francisco.*